Hashing & MST Properties Study Guide

Hash Functions, Collision Resolution, Open Addressing, and Minimum Spanning Tree Properties

Complete Reference for Examination Preparation

Comprehensive Analysis with Theorems, Proofs & Viva Q&A

STUDY MATERIAL

Table of Contents

1. Hashing Fundamentals

1.1 Introduction to Hashing

1.2 Static vs Random Hashing

1.3 Independent Uniform Hashing Assumptions

1.4 Viva Questions & Answers

2. Hash Function Methods

2.1 Division Method

2.2 Multiplication Method

2.3 Multiply-Shift Method

2.4 Comparison of Hash Methods

2.5 Viva Questions & Answers

3. Collision Resolution Strategies

3.1 Chaining vs Open Addressing

3.2 Chaining Mechanism

3.3 Viva Questions & Answers

4. Open Addressing & Probing Techniques

4.1 Linear Probing

4.2 Quadratic Probing

4.3 Double Hashing

4.4 Proof: Double Hashing Covers All Slots

4.5 Viva Questions & Answers

5. Hash Table Operations & Deletion

5.1 Insertion Operation

5.2 Search Operation

5.3 Deletion Strategies

5.4 Viva Questions & Answers

6. MST Properties

6.1 Cut Property

6.2 Cycle Property

6.3 Viva Questions & Answers

1. Hashing Fundamentals

1.1 Introduction to Hashing

Hashing is a technique that maps keys to values using a **hash function**. A hash table is a data structure that implements an associative array, allowing average-case $O(1)$ time complexity for insertions, deletions, and lookups.

> **Core Idea:** Given a universe of keys U and a hash table of size m, a hash function $h : U \to \{0, 1, \ldots, m-1\}$ maps each key to a slot (bucket) in the table.

When two different keys map to the same slot, a **collision** occurs. Resolving collisions efficiently is central to hash table design. The two main strategies are:

- **Chaining:** Each slot stores a linked list of all keys that hash to it
- **Open Addressing:** All keys are stored in the table itself; collisions are resolved by probing for alternative slots

1.2 Static vs Random Hashing

Hash functions fall into two broad categories based on how they are selected:

Aspect	Static Hashing	Random Hashing
Definition	Hash function is fixed and does not change with the input	Hash function is chosen randomly from a family of hash functions
Universality	Not universal; may not evenly distribute keys	Universal hash families guarantee low collision probability
Performance	Deterministic; may perform poorly on certain key sets	Probabilistic guarantees; good performance on average
Use Case	Simple implementations, known key distributions	Robust against adversarial inputs, theoretical analysis
Examples	Division method, Multiplication method	Universal hash families, Multiply-Shift

1.3 Independent Uniform Hashing Assumptions

For theoretical analysis of hash tables, we use the **Simple Uniform Hashing Assumption (SUHA)**:

Simple Uniform Hashing Assumption

The hash function h satisfies:

1. **Uniformity:** Each key is equally likely to hash to any of the m slots, i.e., $P(h(k) = i) = \frac{1}{m}$ for all $i \in \{0, 1, \ldots, m-1\}$.
2. **Independence:** The hash values of different keys are independent random variables.

Important: In practice, truly uniform and independent hash functions do not exist for arbitrary key sets. SUHA is a theoretical model used to analyze expected performance. **Universal hashing** provides a practical way to approximate these assumptions by randomly selecting a hash function from a carefully designed family.

1.4 Viva Questions & Answers

Viva Q&A: Hashing Fundamentals

Q1: What is a hash function and what is its purpose?

A hash function h maps keys from a large universe U to indices $\{0, 1, \ldots, m-1\}$ of a hash table. Its purpose is to enable $O(1)$ average-case time for insert, delete, and search operations by directly computing the storage location from the key.

Q2: What is the difference between static and random hashing?

Static hashing uses a fixed hash function that never changes, which can lead to poor performance if the key distribution happens to create many collisions. Random hashing selects a hash function randomly from a family, providing probabilistic guarantees and protection against adversarial inputs that could force worst-case behavior.

Q3: What are the Simple Uniform Hashing Assumptions?

(1) **Uniformity:** Every key is equally likely to hash to any slot, with probability $1/m$. (2) **Independence:** Hash values of different keys are independent of each other. These assumptions allow us to prove that chaining gives $O(1+\alpha)$ expected search time, where $\alpha = n/m$ is the load factor.

Q4: Why is the assumption of independence important for hashing analysis?

Independence ensures that the hash value of one key does not influence the hash value of another. Without this, clustered or correlated keys could all map to the same region of the table, causing unexpectedly long chains or probe sequences and degrading performance.

Q5: Can we achieve true uniform hashing in practice?

No, not for arbitrary keys. Truly random hash functions would require exponentially large space to represent. In practice, we use **universal hash families**, which guarantee that for any two distinct keys, the probability of collision is at most $1/m$.

2. Hash Function Methods

Three classical methods for constructing hash functions are the Division Method, Multiplication Method, and Multiply-Shift Method. Each has different trade-offs in terms of simplicity, speed, and distribution quality.

2.1 Division Method

Division Method

$$h(k) = k \mod m$$

where m is the number of slots in the hash table.

This is the simplest hash function. It computes the remainder when the key k is divided by the table size m.

Advantage	Disadvantage
Very fast: single modulo operation	Sensitive to choice of m
Easy to implement	Poor distribution if m shares factors with keys

Critical Choice: Do NOT choose m as a power of 2 (e.g., $m = 2^p$). If $m = 2^p$, then $h(k)$ depends only on the lowest p bits of k, ignoring all higher bits. This causes collisions when keys share the same low bits.

Best practice: Choose m to be a prime number not too close to a power of 2.

Example

For $m = 11$ (a small prime) and $k = 31$:

$$h(31) = 31 \mod 11 = 9$$

2.2 Multiplication Method

Multiplication Method

$$h(k) = \lfloor m \cdot (kA \mod 1) \rfloor$$

where A is a constant in the range $0 < A < 1$, and $(kA \mod 1)$ means the fractional part of kA.

This method multiplies the key by a constant, takes the fractional part, then scales by m and floors the result.

Advantage	Disadvantage
Works well with any table size m (including powers of 2)	Slightly slower than division (multiplication + fractional extraction)
Spreads keys more uniformly across all slots	Choice of A affects quality; good values needed

Good Choice of A: Knuth suggests $A \approx (\sqrt{5} - 1)/2 \approx 0.6180339887\ldots$ (the golden ratio conjugate). This irrational number tends to spread keys uniformly.

Example

For $m = 8$, $A = 0.618$, $k = 31$:

$$kA = 31 \times 0.618 = 19.158$$

$$kA \mod 1 = 0.158$$

$$h(31) = \lfloor 8 \times 0.158 \rfloor = \lfloor 1.264 \rfloor = 1$$

2.3 Multiply-Shift Method

Multiply-Shift Method

Treat the key k and a random odd constant a as w-bit integers. Compute:

$$h(k) = (a \cdot k) \gg (w - p)$$

where $m = 2^p$ (table size is a power of 2), and $\gg$ denotes right bit shift.

This method uses bitwise operations for maximum speed on modern processors. It extracts the top p bits from the product $a \cdot k$.

Advantage	Disadvantage
Extremely fast: single multiply + shift	Requires table size to be a power of 2
Single CPU instruction on most architectures	Need to carefully choose random odd a
Part of 2-independent hash families (strong theoretical guarantees)	2-independence may not be enough for all applications

2.4 Comparison of Hash Methods

Feature	Division	Multiplication	Multiply-Shift
Formula	$k \mod m$	$\lfloor m(kA \mod 1) \rfloor$	$(a \cdot k) \gg (w - p)$
Table Size m	Best: prime	Any value	Must be power of 2
Speed	Fast	Moderate	Very fast
Key Dependency	All bits of k	All bits (via multiplication)	All bits (via multiplication)
Independence Level	None (static)	None (static)	2-independent

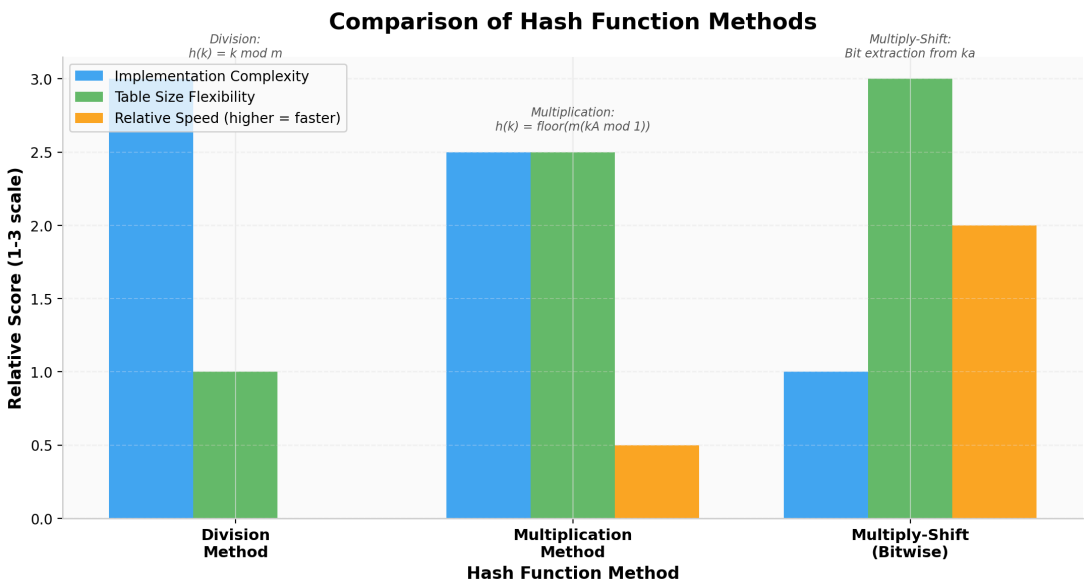

Figure 1: *Comparison of hash function methods across implementation complexity, table size flexibility, and speed*

Viva Q&A: Hash Function Methods

Q1: Write the hash function for the Division Method.

$h(k) = k \mod m$, where m is the table size. The key k is divided by m and the remainder is the hash value.

Q2: What value of m should be avoided in the Division Method, and why?

Powers of 2 (e.g., $m = 2^p$) should be avoided. If $m = 2^p$, then $h(k) = k \mod 2^p$ depends only on the lowest p bits of k, completely ignoring all higher-order bits. This causes many collisions when keys differ only in their high bits.

Q3: Write the hash function for the Multiplication Method.

$h(k) = \lfloor m \cdot (kA \mod 1) \rfloor$, where A is a constant with $0 < A < 1$. The term $(kA \mod 1)$ extracts the fractional part of kA, which is then scaled by m and floored.

Q4: Why is the Multiplication Method less sensitive to the choice of m?

Unlike the Division Method, the Multiplication Method spreads the influence of all bits of k across the entire hash value through the multiplication step. The choice of m does not create the same systematic bias because the fractional part extraction distributes keys more uniformly regardless of m.

Q5: Explain the Multiply-Shift Method and its key advantage.

$h(k) = (a \cdot k) \gg (w - p)$ where $m = 2^p$ and a is a random odd constant. The key advantage is speed: it requires only a single multiplication and a bit shift, which together execute in just 2-3 CPU cycles on modern hardware. It is also part of 2-independent hash families, providing strong theoretical guarantees.

Q6: Why must the constant a be odd in Multiply-Shift?

If a were even, then $a \cdot k$ would always be even, meaning the least significant bit would always be 0, severely limiting the hash distribution. An odd a ensures the multiplication is a bijection (permutation) in the w-bit space, preserving the randomness of key bits in the product.

3. Collision Resolution Strategies

When two keys hash to the same slot, a collision occurs. The two primary approaches to handling collisions are **Chaining** (storing colliding keys outside the table) and **Open Addressing** (finding another slot within the table).

3.1 Chaining vs Open Addressing

Feature	Chaining	Open Addressing
Storage	Colliding keys stored in linked lists outside the table	All keys stored within the table; alternative slots probed
Extra Space	$O(n)$ for pointers and nodes	No extra space beyond the table
Load Factor α	Can exceed 1 ($\alpha = n/m$, unbounded)	Must be $\alpha \leq 1$ (cannot store more keys than slots)
Cache Performance	Poor: linked lists scatter memory accesses	Better: all data in contiguous array
Deletion	Simple: remove from linked list	Complex: requires special handling (tombstones)
Clustering	No clustering within the table	Primary and secondary clustering possible
Expected Search	$O(1 + \alpha)$	$O(1/(1 - \alpha))$ for uniform probing

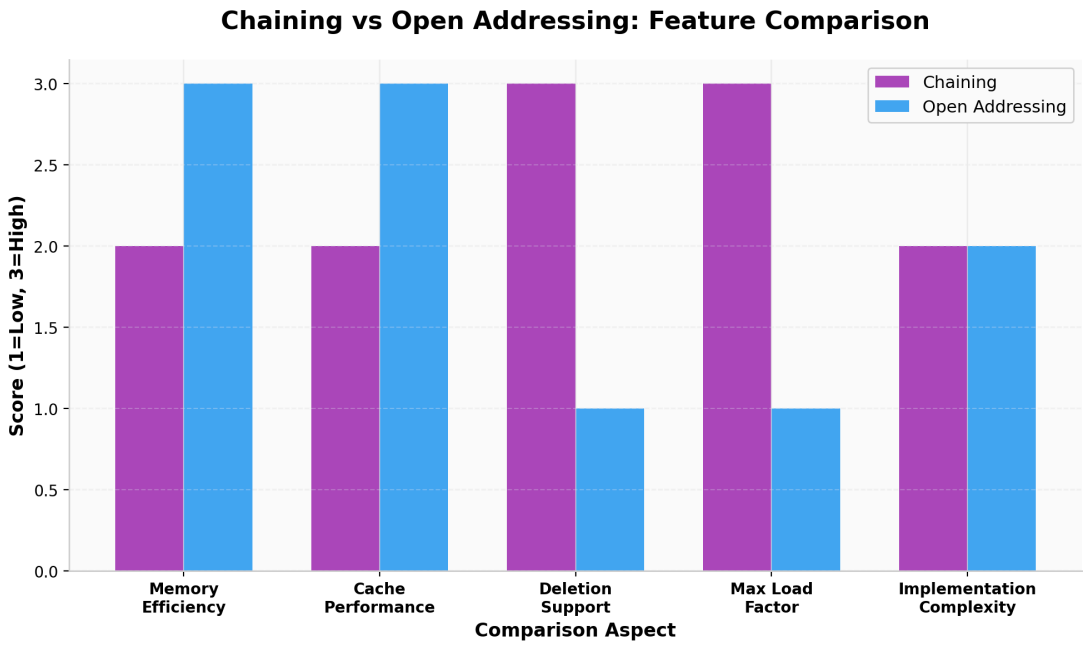

Figure 2: *Feature comparison between Chaining and Open Addressing collision resolution strategies*

3.2 Chaining Mechanism

In chaining, each slot of the hash table contains a pointer to a linked list (or another data structure). All keys that hash to the same slot are inserted into that slot's linked list.

Analysis of Chaining

Under SUHA, the expected length of each chain is $\alpha = n/m$ (the load factor). Therefore:

- **Successful search:** $O(1 + \alpha)$ — on average, α elements to check in the chain
- **Unsuccessful search:** $O(1 + \alpha)$ — must traverse the entire chain
- **Insertion:** $O(1)$ if inserted at the head of the list

When $\alpha = O(1)$ (i.e., $n = O(m)$), all operations take $O(1)$ expected time.

Probing Sequence in Open Addressing

In open addressing, the **probe sequence** is a permutation of $\{0, 1, \ldots, m - 1\}$. For a key k, the hash function generates a sequence of slots to try:

$$\langle h(k, 0), h(k, 1), \ldots, h(k, m - 1) \rangle$$

Types of Probing

Probing Type	Probe Function	Key Characteristic
Linear Probing	$h(k,i) = (h'(k) + i) \mod m$	Check next consecutive slot; suffers from primary clustering
Quadratic Probing	$h(k,i) = (h'(k) + c_1 i + c_2 i^2) \mod m$	Quadratic gap between probes; avoids primary clustering
Double Hashing	$h(k,i) = (h_1(k) + i \cdot h_2(k)) \mod m$	Uses two hash functions; best distribution among open addressing methods

3.3 Viva Questions & Answers

Viva Q&A: Collision Resolution

Q1: What is a collision in hashing?

A collision occurs when two different keys $k_1 \neq k_2$ hash to the same slot: $h(k_1) = h(k_2)$. Since the universe of keys is typically much larger than the table size ($|U| \gg m$), collisions are inevitable.

Q2: What are the two main collision resolution strategies?

Chaining: Each slot stores a linked list of all keys that hash to it. Colliding keys are stored outside the main table. **Open Addressing:** All keys are stored in the table itself. When a collision occurs, alternative slots are probed until an empty slot is found.

Q3: What is the load factor α, and why is it important?

$\alpha = n/m$ where n is the number of keys and m is the table size. For chaining, the expected chain length is α, so search time is $O(1 + \alpha)$. For open addressing, performance degrades as $\alpha \to 1$ because fewer empty slots remain for probing.

Q4: Can the load factor exceed 1 in chaining? In open addressing?

In **chaining**, yes: α can exceed 1 since chains can grow arbitrarily long. In **open addressing**, no: $\alpha \leq 1$ always, since there cannot be more keys than table slots. When $\alpha \approx 1$ in open addressing, insertion fails.

Q5: What is a probe sequence in open addressing?

A probe sequence $\langle h(k,0), h(k,1), ..., h(k,m-1)\rangle$ is an ordering of table slots that is examined when inserting or searching for key k. It should be a permutation of $\{0, 1, ..., m-1\}$ to ensure all slots are eventually checked.

4. Open Addressing & Probing Techniques

Open addressing resolves collisions by searching for alternative empty slots within the hash table itself. The method used to search for these alternative slots is called **probing**. Three common probing techniques are linear probing, quadratic probing, and double hashing.

4.1 Linear Probing

Linear Probing

$$h(k, i) = (h'(k) + i) \mod m$$

where $h'(k)$ is an ordinary hash function and $i = 0, 1, 2, \ldots$ is the probe number.

Linear probing checks consecutive slots until an empty one is found. While simple and cache-friendly, it suffers from **primary clustering** — contiguous blocks of occupied slots form, making future insertions into those regions increasingly expensive.

Advantage	Disadvantage
Simple to implement	Primary clustering: long runs of occupied slots form
Excellent cache locality (consecutive memory access)	Search time degrades significantly as α increases
Fastest probing method in practice for low α	Expected probes: $\approx \frac{1}{2}(1 + \frac{1}{(1-\alpha)^2})$ for unsuccessful search

4.2 Quadratic Probing

Quadratic Probing

$$h(k, i) = (h'(k) + c_1 i + c_2 i^2) \mod m$$

where c_1 and c_2 are constants (commonly $c_1 = c_2 = \frac{1}{2}$, giving triangular numbers).

Quadratic probing uses a quadratic function of the probe number to determine the next slot to examine. This eliminates primary clustering because keys with the same initial hash value follow different probe sequences.

Secondary Clustering: Quadratic probing still suffers from secondary clustering: if two keys have the same initial hash value, they follow the same probe sequence. Also, if m is not chosen carefully, the probe sequence may not cover all slots.

4.3 Double Hashing

Double Hashing

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \mod m$$

where $h_1(k)$ and $h_2(k)$ are two independent hash functions.

Double hashing uses a second hash function to determine the step size for probing. Each key has its own probe step size $h_2(k)$, which nearly eliminates both primary and secondary clustering.

Advantage	Disadvantage
Best distribution: eliminates both primary and secondary clustering	Two hash functions must be computed per key
Probe sequence depends on key in two independent ways	Slightly more complex implementation
Expected probes: $\approx \frac{1}{1-\alpha}$ for uniform probing	$h_2(k)$ must be relatively prime to m for full coverage

Ensuring Full Coverage: To guarantee the probe sequence covers all m slots, $\gcd(h_2(k), m) = 1$ must hold. Common choices: (1) m is prime and $h_2(k) \in \{1, \ldots, m-1\}$; (2) $m = 2^p$ and $h_2(k)$ is odd.

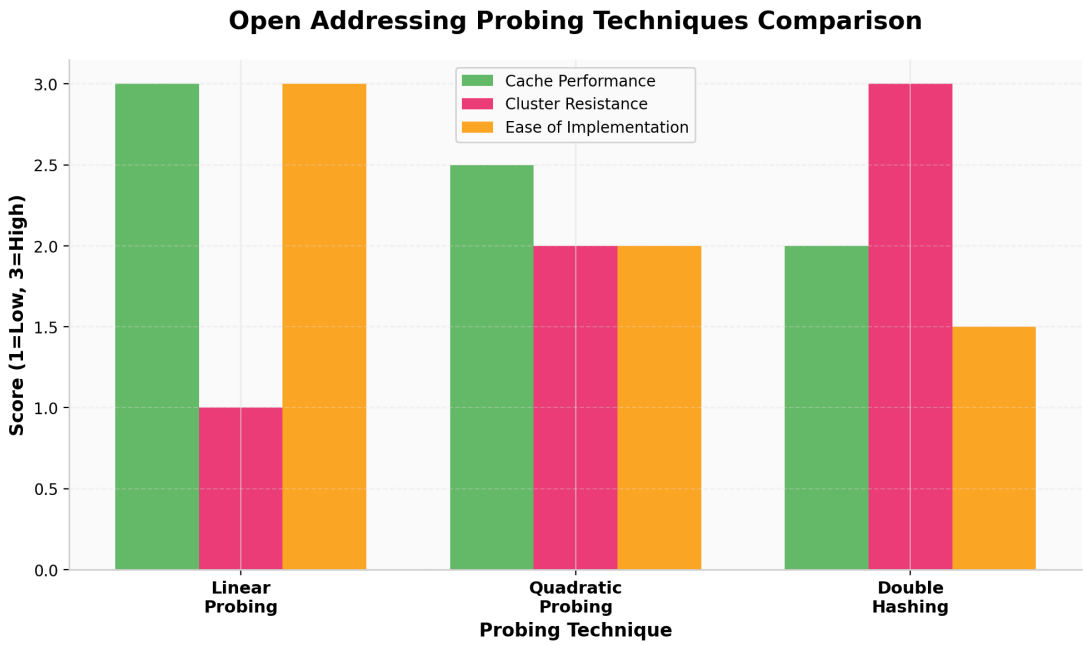

Figure 3: *Comparison of linear probing, quadratic probing, and double hashing across cache performance, cluster resistance, and implementation ease*

4.4 Proof: Double Hashing Covers All Slots

Theorem: Probe Sequence Coverage

If $\gcd(h_2(k), m) = 1$, then the double hashing probe sequence $h(k, i) = (h_1(k) + i \cdot h_2(k)) \mod m$ generates a permutation of $\{0, 1, \ldots, m-1\}$.

Proof: Suppose two probe numbers i and j (with $0 \leq i < j < m$) yield the same slot:

$$(h_1(k) + i \cdot h_2(k)) \equiv (h_1(k) + j \cdot h_2(k)) \pmod{m}$$

Subtracting $h_1(k)$ from both sides:

$$i \cdot h_2(k) \equiv j \cdot h_2(k) \pmod{m}$$

$$(j - i) \cdot h_2(k) \equiv 0 \pmod{m}$$

This means m divides $(j - i) \cdot h_2(k)$. Since $\gcd(h_2(k), m) = 1$, it follows that m must divide $(j - i)$. But $0 < j - i < m$, which is a contradiction. Therefore, all m probe positions generate distinct slots, forming a complete permutation.

Viva Q&A: Open Addressing & Probing

Q1: What is the probe function for linear probing?

$h(k, i) = (h'(k) + i) \mod m$. It checks slots $h'(k), h'(k) + 1, h'(k) + 2, \ldots$ (modulo m) until an empty slot is found.

Q2: What is primary clustering, and which probing method suffers from it?

Primary clustering is the formation of long contiguous blocks of occupied slots. When a slot in a cluster is targeted, the probe sequence traverses the entire cluster, making insertions and searches slower. **Linear probing** suffers from primary clustering because keys hashing near each other follow the same linear sequence.

Q3: Write the probe function for quadratic probing.

$h(k, i) = (h'(k) + c_1 i + c_2 i^2) \mod m$. A common simplification uses $h(k, i) = (h'(k) + \frac{i(i+1)}{2}) \mod m$, which generates triangular numbers as offsets, avoiding primary clustering.

Q4: Write the probe function for double hashing.

$h(k, i) = (h_1(k) + i \cdot h_2(k)) \mod m$, where $h_1(k)$ is the primary hash function and $h_2(k)$ is the secondary hash function that determines the step size. Each key has its own step size, virtually eliminating clustering.

Q5: Prove that double hashing covers all table slots if $\gcd(h_2(k), m) = 1$.

Assume two probes $i \neq j$ give the same slot. Then $(j - i) \cdot h_2(k) \equiv 0 \pmod{m}$. Since $\gcd(h_2(k), m) = 1$, m must divide $(j - i)$. But $0 < |j - i| < m$, a contradiction. Hence all m probes produce distinct slots — a complete permutation.

Q6: How does the expected search time compare between linear probing and double hashing?

For unsuccessful search at load factor α: Linear probing takes $\approx \frac{1}{2}(1 + \frac{1}{(1-\alpha)^2})$ probes. Double hashing takes $\approx \frac{1}{1-\alpha}$ probes. Linear probing degrades much faster as $\alpha \to 1$ due to clustering. At $\alpha = 0.9$, linear probing needs ~50 probes while double hashing needs ~10.

5. Hash Table Operations & Deletion

5.1 Insertion Operation

Chaining Insertion

1. Compute $h(k)$ to find the target slot
2. Insert the key at the head of the linked list at that slot
3. Time complexity: $O(1)$

Open Addressing Insertion

1. Compute the probe sequence $h(k, 0), h(k, 1), \ldots$
2. For each slot in the sequence: if empty or marked deleted, insert the key
3. If all m slots are full (occupied), insertion fails (table overflow)
4. Time complexity: $O(1/(1-\alpha))$ expected

5.2 Search Operation

Chaining Search

1. Compute $h(k)$ to find the target slot
2. Traverse the linked list at that slot, comparing each node with k
3. If found, return the node; otherwise, key is not in the table
4. Expected time: $O(1+\alpha)$

Open Addressing Search

1. Compute the probe sequence $h(k, 0), h(k, 1), \ldots$
2. For each slot: if it contains k, return found; if empty, key is not present
3. If a deleted (tombstone) slot is encountered, continue probing
4. Expected time: $O(1/(1-\alpha))$ for uniform/double hashing

5.3 Deletion Strategies

Deletion is straightforward in chaining (remove from the linked list) but problematic in open addressing. If a key is simply removed, it creates a gap that could break probe sequences for other keys.

Problem with Direct Deletion in Open Addressing

Consider: key A was inserted at slot 5 after probing. Later, key B (which hashes to the same initial slot) was inserted at slot 6 after finding slot 5 occupied. If we delete A from slot 5, making it empty, a search for B would stop at slot 5 (which now appears empty) and incorrectly conclude B is not in the table.

Solution 1: Tombstones (Lazy Deletion)

Instead of clearing the slot, mark it with a special **tombstone** (or "deleted") marker. During search, tombstones are treated as occupied (probing continues). During insertion, tombstones are treated as empty (new keys can be inserted there).

Slot State	Search Behavior	Insertion Behavior
Occupied	Compare key; if match, found; else continue	Continue probing
Tombstone (Deleted)	Continue probing (key might be further ahead)	Insert here (slot reusable)
Empty	Stop: key not present	Insert here

Solution 2: Rehashing

When the number of tombstones exceeds a threshold, rebuild the entire hash table by re-inserting all remaining keys. This removes all tombstones and restores optimal performance.

> **Tombstone Accumulation:** Over many deletions, tombstones can accumulate and degrade search performance (searches must traverse through them). Periodic rehashing is necessary to maintain $O(1)$ operations.

5.4 Viva Questions & Answers

Viva Q&A: Hash Table Operations & Deletion

Q1: Why is deletion problematic in open addressing?

If a key is simply removed, its slot becomes empty, which can break probe sequences for other keys. During a search, encountering an empty slot normally means "stop searching." But if that slot previously contained a key that was part of another key's probe path, the search would incorrectly terminate before finding the target.

Q2: What is a tombstone in hash tables?

A tombstone is a special marker placed in a slot when a key is deleted (instead of leaving the slot truly empty). During **search**, tombstones are treated as occupied so probing continues past them. During **insertion**, tombstones are treated as empty so new keys can reuse the slot.

Q3: What problem arises from too many tombstones?

Tombstones accumulate over time and slow down searches because the probe sequence must traverse through them as if they were occupied keys. When tombstones exceed a threshold (e.g., $\frac{1}{4}$ of the table), the table should be **rehashed**: all non-deleted keys are extracted and re-inserted into a fresh table.

Q4: What is rehashing, and when is it triggered?

Rehashing is rebuilding the hash table by extracting all non-deleted keys and inserting them into a new, larger table. It is triggered when the load factor exceeds a threshold (typically $\alpha > 0.75$) or when too many tombstones have accumulated. Rehashing removes all tombstones and restores $O(1)$ expected performance.

Q5: Compare the time complexity of search in chaining vs open addressing.

Chaining:$O(1+\alpha)$ expected, where $\alpha = n/m$ can exceed 1. **Open addressing:**$O(1/(1-\alpha))$ expected for uniform probing/double hashing, and $\approx \frac{1}{2}(1+1/(1-\alpha)^2)$ for linear probing. At $\alpha = 0.5$, both take $O(1)$. At $\alpha = 0.9$, chaining takes $O(1.9)$ while open addressing takes $O(10)$.

6. MST Properties

The **Minimum Spanning Tree (MST)** of a connected, weighted, undirected graph is a subset of edges that connects all vertices with the minimum total edge weight. Two fundamental properties govern which edges can be part of an MST: the **Cut Property** and the **Cycle Property**.

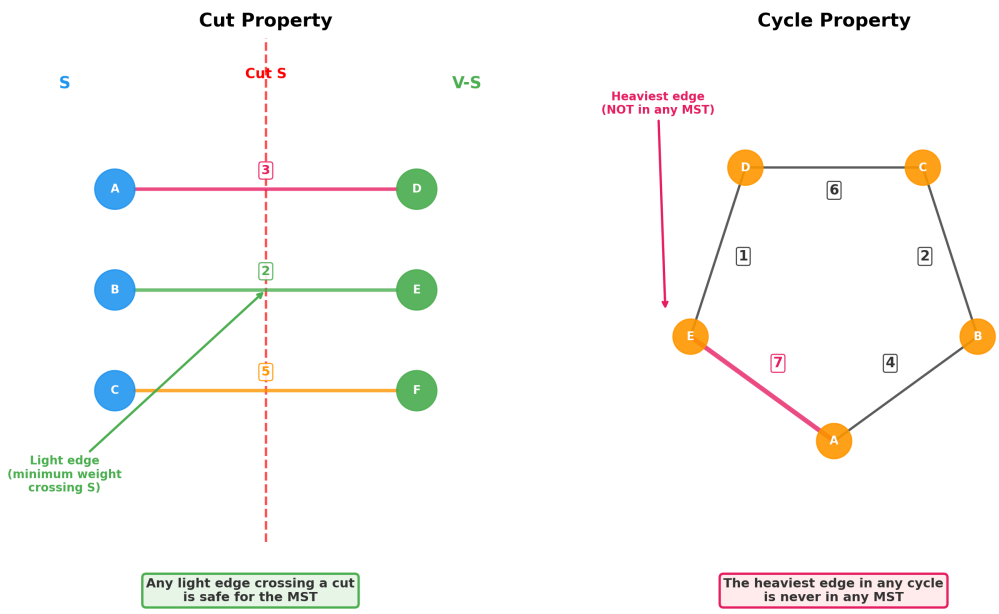

Figure 4: *Visual illustration of the Cut Property (light edge crossing a cut is safe) and Cycle Property (heaviest edge in a cycle is never in any MST)*

6.1 Cut Property

Cut Property

Let S be any subset of vertices (a **cut**), and let e be the minimum-weight edge with exactly one endpoint in S and one endpoint in $V \setminus S$ (a **light edge** crossing the cut). Then e belongs to *some* MST of the graph.

Proof of the Cut Property

Let T be an MST of the graph. If T already contains e, we are done. If not:

1. Since T is a spanning tree, it must contain at least one edge e' that crosses the cut $(S, V \setminus S)$ (otherwise T would be disconnected).

2. Consider $T' = T \cup \{e\} \setminus \{e'\}$. Adding e to T creates a cycle; removing e' breaks the cycle and restores a tree.
3. Since e is a light edge (minimum weight crossing the cut), $w(e) \leq w(e')$.
4. Therefore, $w(T') = w(T) + w(e) - w(e') \leq w(T)$.
5. Since T is an MST, $w(T') = w(T)$, so T' is also an MST containing e.

> **Intuition:** The cheapest way to connect the two sides of any partition (cut) must be used in an optimal spanning tree. If you didn't use the cheapest crossing edge, you could swap it in and get an equally good or better tree.

Significance

The Cut Property is the theoretical foundation for both Prim's and Kruskal's algorithms:

- **Prim's algorithm** repeatedly adds the light edge crossing the cut defined by the current tree and the remaining vertices
- **Kruskal's algorithm** adds edges in increasing weight; each added edge is a light edge for some cut

6.2 Cycle Property

> **Cycle Property**
>
> Let C be any **cycle** in the graph, and let e be the **maximum-weight edge** in C. Then e does *not* belong to *any* MST of the graph.

Proof of the Cycle Property

Suppose for contradiction that e (the heaviest edge in cycle C) is in some MST T:

1. Remove e from T. This disconnects T into two components, creating a cut.
2. The cycle C must contain another edge e' that crosses this same cut (since a cycle connects back to its starting point).
3. Since e is the maximum-weight edge in C, we have $w(e') < w(e)$ (or $w(e') \leq w(e)$ if not unique).
4. Consider $T' = T \setminus \{e\} \cup \{e'\}$. T' is connected, has $n - 1$ edges, and is therefore a spanning tree.
5. $w(T') = w(T) - w(e) + w(e') < w(T)$, contradicting the minimality of T.

Intuition: In any cycle, the heaviest edge is never needed. You can always replace it with a lighter edge from the same cycle and still maintain connectivity with lower total weight.

Comparison of Cut and Cycle Properties

Aspect	Cut Property	Cycle Property
Statement	Light edge crossing any cut is in *some* MST	Heaviest edge in any cycle is in *no* MST
Quantifier on MST	Some MST (safe edge)	No MST (unsafe edge)
Use in Algorithms	Prim's: always adds safe edges	Kruskal's: skips edges that would form cycles
Action	Include the edge	Exclude the edge
Proof Technique	Exchange argument (swap in lighter edge)	Exchange argument (swap out heavier edge)

6.3 Viva Questions & Answers

Viva Q&A: MST Properties

Q1: State and prove the Cut Property for MSTs.

Statement: The minimum-weight edge crossing any cut $(S, V \setminus S)$ belongs to some MST. **Proof:** Let e be the light crossing edge and T an MST. If $e \notin T$, some edge $e' \in T$ also crosses the cut. Swapping e' for e gives a spanning tree T' with $w(T') \leq w(T)$. Since T is minimum, T' is also an MST containing e.

Q2: State and prove the Cycle Property for MSTs.

Statement: The maximum-weight edge in any cycle does not belong to any MST. **Proof:** Assume heaviest edge e in cycle C is in MST T. Remove e from T, disconnecting it. Since C is a cycle, some other edge $e' \in C$ reconnects the components. Since $w(e') < w(e)$, the new tree $T' = T \setminus \{e\} \cup \{e'\}$ is lighter than T, contradicting minimality.

Q3: How does the Cut Property justify Prim's algorithm?

Prim's algorithm maintains a growing tree S. At each step, it adds the minimum-weight edge connecting S to $V \setminus S$. By the Cut Property, this edge (the light edge crossing cut $(S, V \setminus S)$) is safe — it belongs to some MST. Repeating this $n - 1$ times builds an MST.

Q4: How does the Cycle Property justify Kruskal's algorithm?

Kruskal's processes edges in increasing weight order. When considering edge e, if adding it creates a cycle, e is the heaviest edge in that newly formed cycle (since all previously added edges are lighter). By the Cycle Property, e cannot be in any MST, so Kruskal correctly skips it.

Q5: Can the heaviest edge in a graph ever be in the MST?

Yes, if it is a bridge (its removal disconnects the graph). A bridge cannot be part of any cycle, so the Cycle Property does not apply. The MST must include all bridges to maintain connectivity, regardless of their weight.

Q6: What happens if edge weights are not distinct? Do the properties still hold?

Yes, both properties still hold. If multiple edges share the minimum weight crossing a cut, any of them is in some MST (the MST may not be unique). If multiple edges share the maximum weight in a cycle, at least one of them is excluded from every MST. The MST is unique only if all edge weights are distinct.